Artificial Intelligence (CS1360)
Informed search algorithms

Introduction

- Heuristic or informed search exploits additional knowledge about the problem that helps to guide search to more promising paths.
  - can find solutions more efficiently than can an uninformed strategy.

- A heuristic function, *h(n)*, provides an **estimate** of the cost of the cheapest path from a given node *n* to a goal state.
  - Must be **zero** if this node represents a **goal** state.
  - Example: **Straight-line distance** from current location to the goal location in a road navigation problem.

Best-first search

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure
```

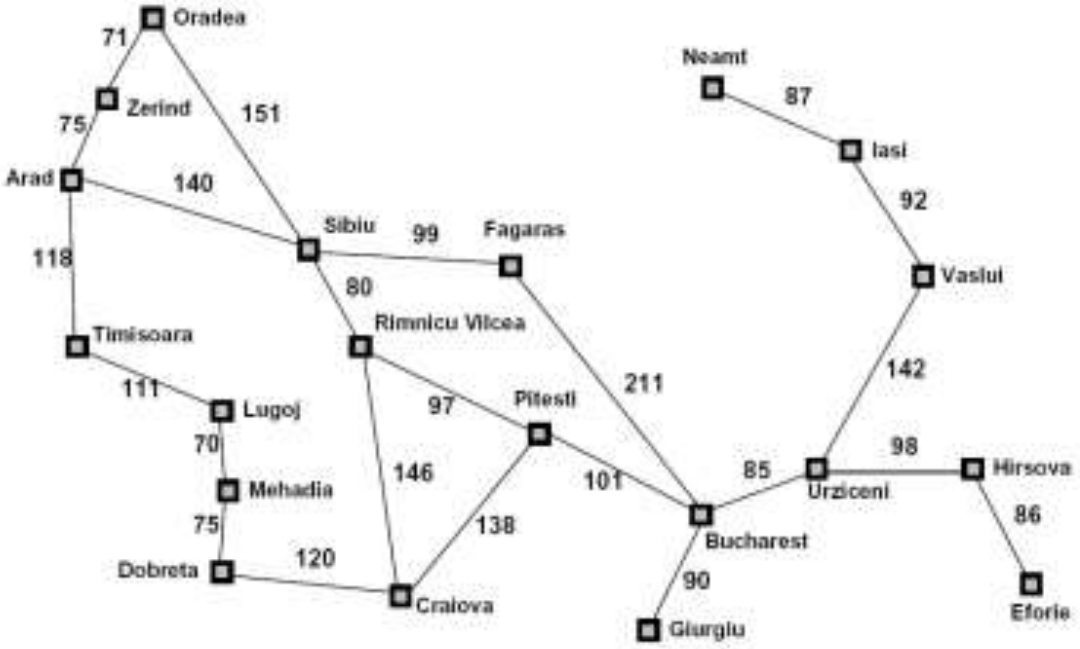
Best-first search

- **Idea:** use an evaluation function *f(n)* for each node
  - Estimate of "desirability"
  - Expand most desirable unexpanded node
- Implementation :
  - Order the nodes in fringe in decreasing order of desirability.
- Special cases:
  - Greedy best-first search
  - A* search
- The performance of heuristic search algorithms depends on the quality of the heuristic function.

Greedy best-first search

- Evaluation function $f(n) = h(n)$
  - *Heuristic* $h(n)$ = estimate of cost from n to goal
  - e.g., $h_{SLD}(n)$ = straight-line distance from n to Bucharest

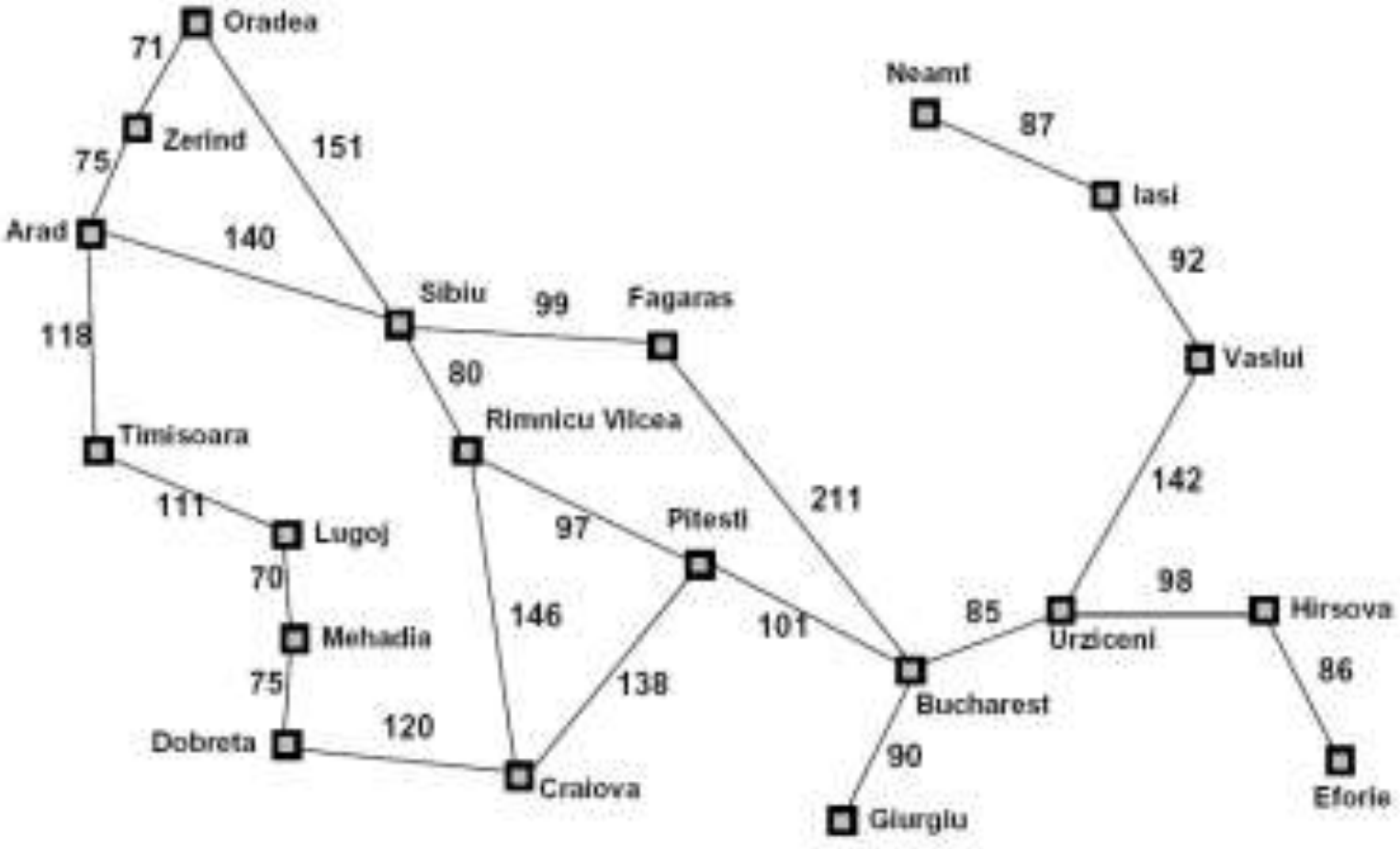
- **Greedy best-first search** expands the node that appears to be closest to goal



$h(n)$

Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Cralova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

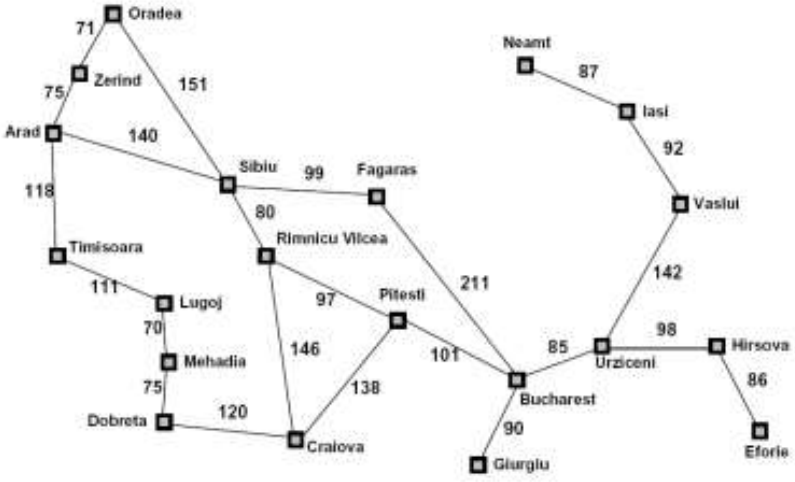
Romania with step costs in km



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

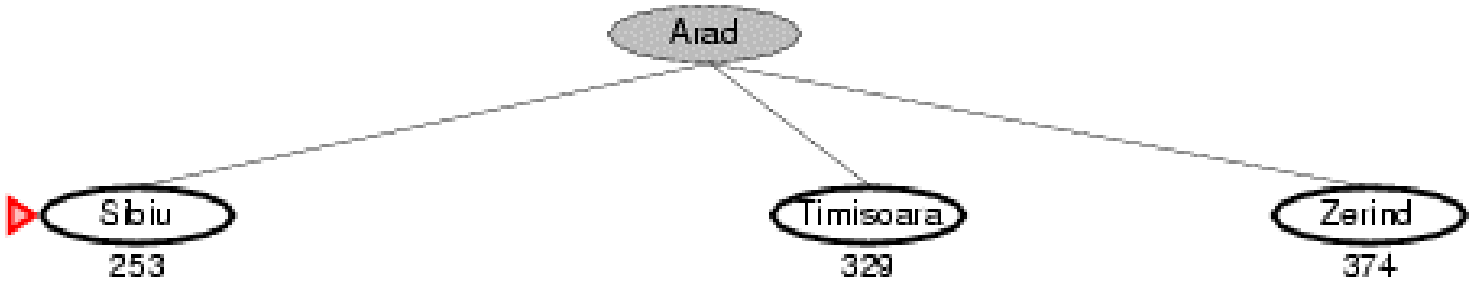
Greedy best-first search example

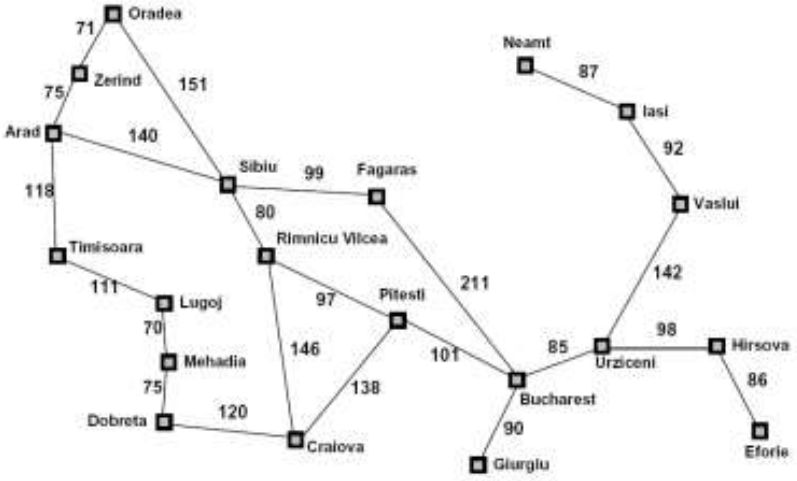




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

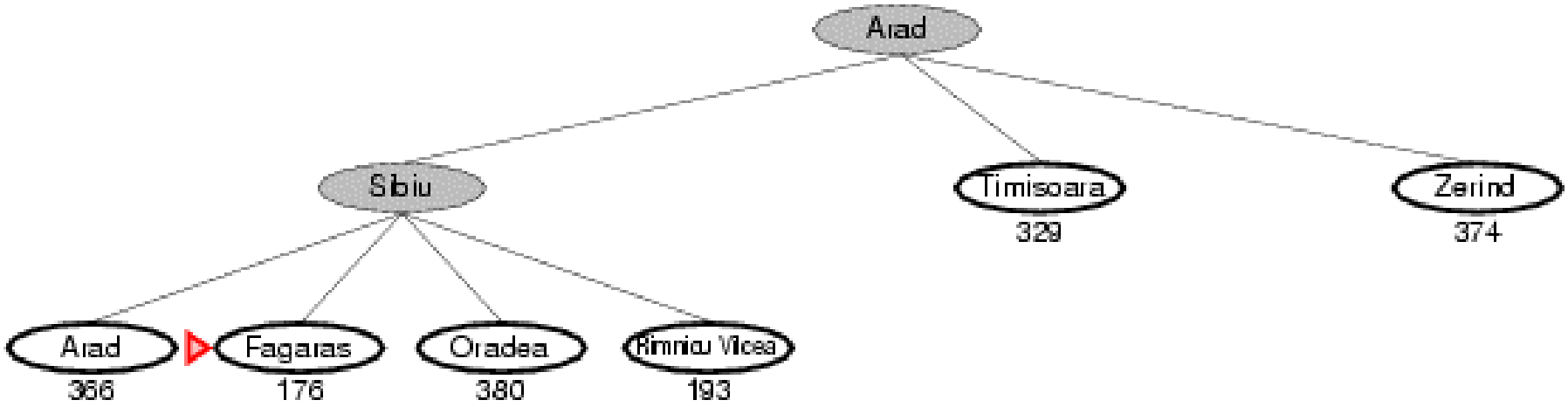
Greedy best-first search example

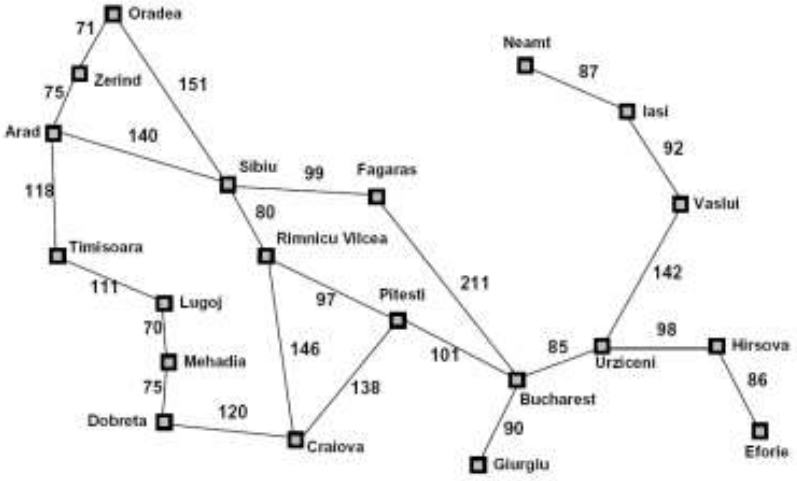




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

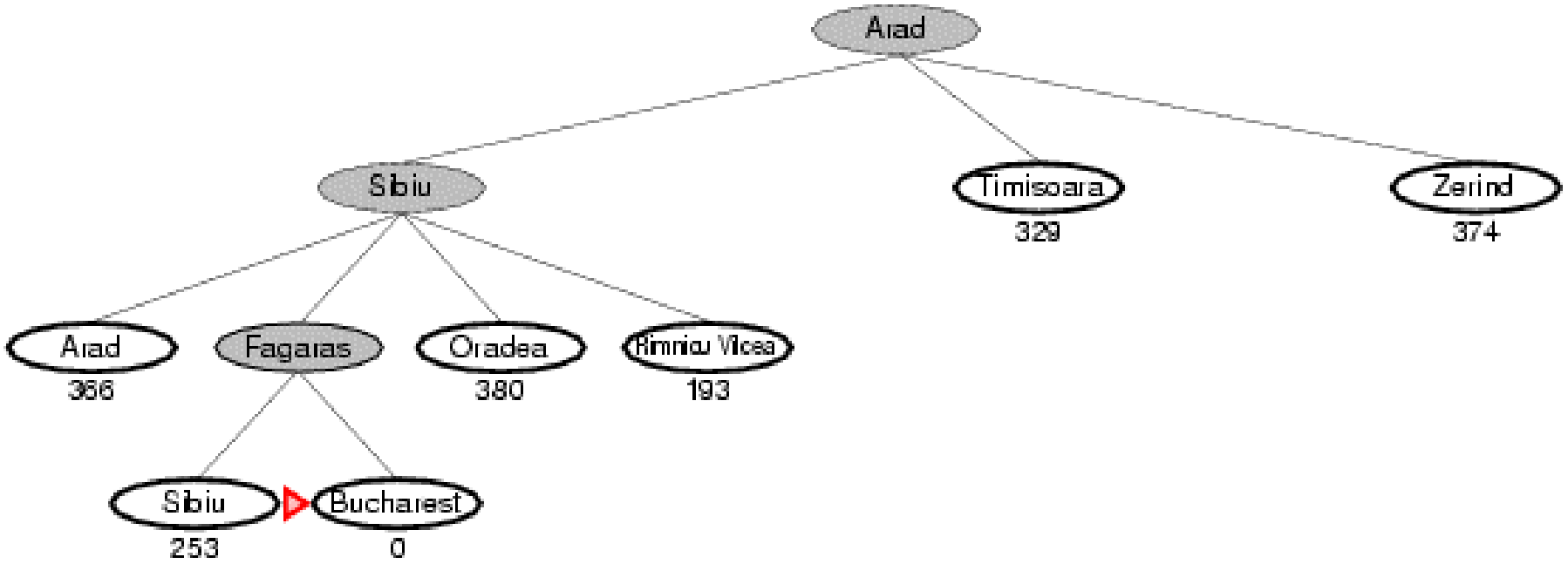
Greedy best-first search example

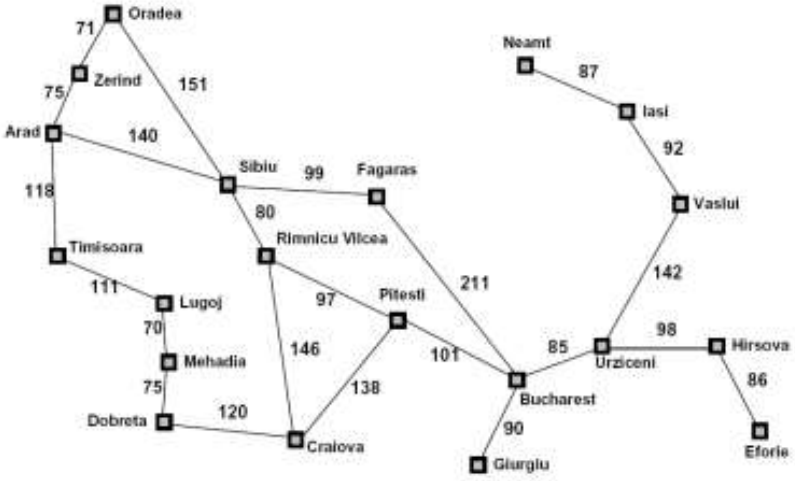




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy best-first search example

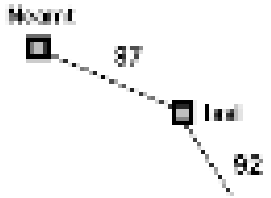




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Properties of greedy best-first search

- **Complete?** Yes in finite state space, but not in infinite state space or tree search as it can get stuck in loops, e.g., Iasi → Neamt → Iasi → Neamt →



- **Optimal?** No
- **Time Complexity**: $O(b^m)$, but a good heuristic can give dramatic improvement
- **Space Complexity:** $O(b^m)$

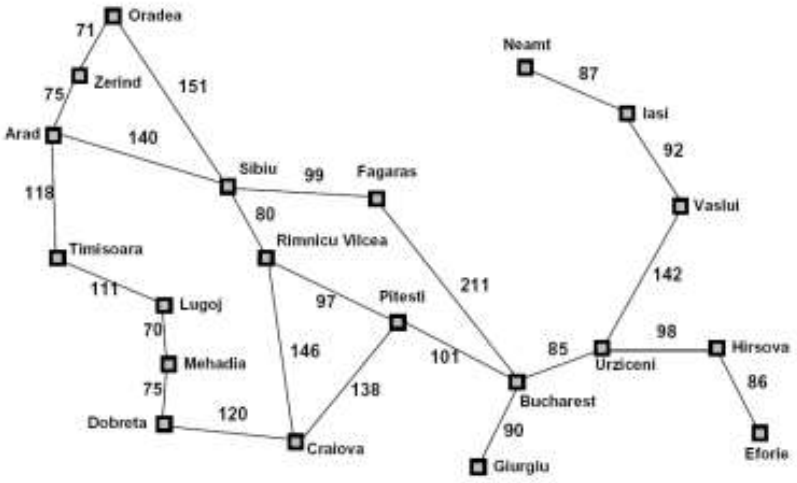
A* search

- **Idea:** avoid expanding paths that are already expensive

  Evaluation function f(n) = g(n) + h(n)
- Combining UCS (g(n)) and Greedy (h(n))
- g(n) = exact cost so far to reach n
- h(n) = estimated cost from n to goal
- f(n) = estimated cost of the best path that continues from n to a goal

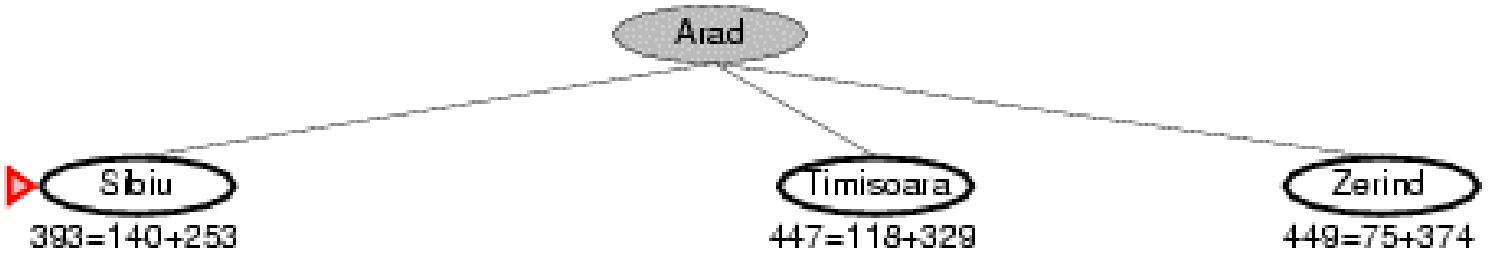
A* search example

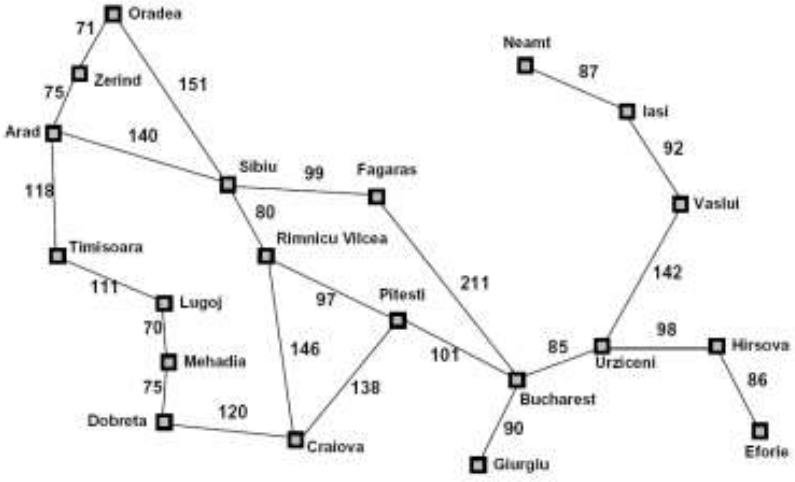




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

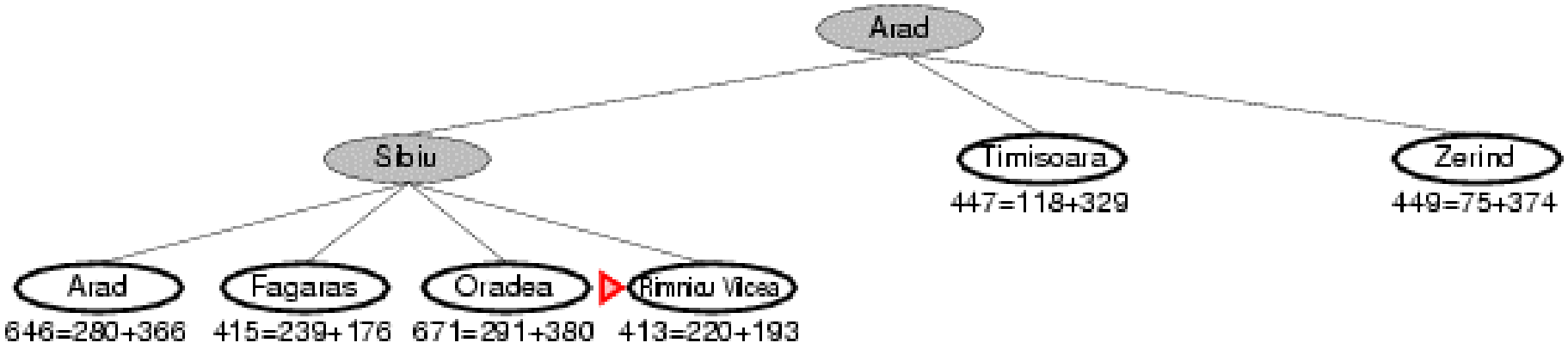
A* search example

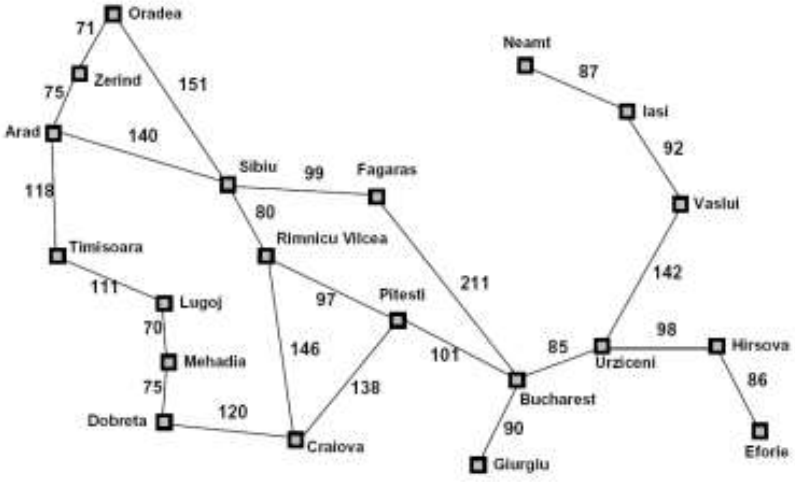




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

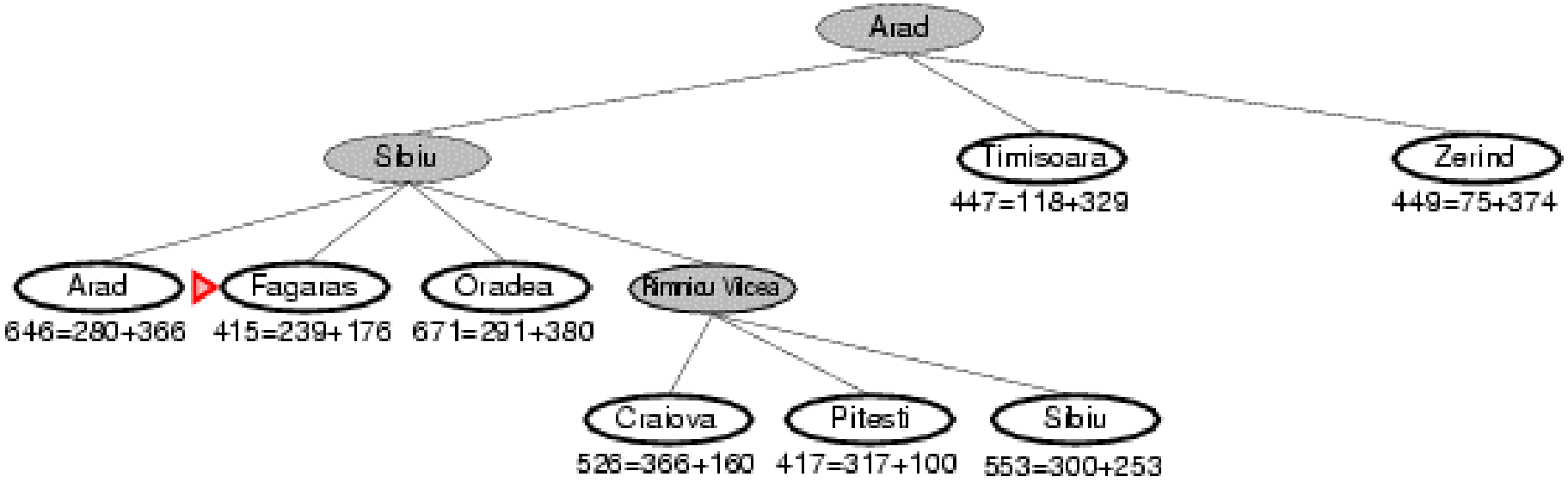
A* search example

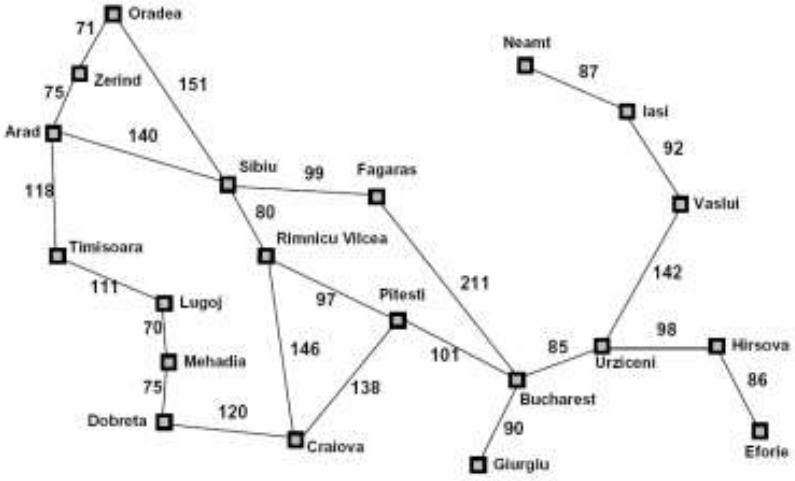




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

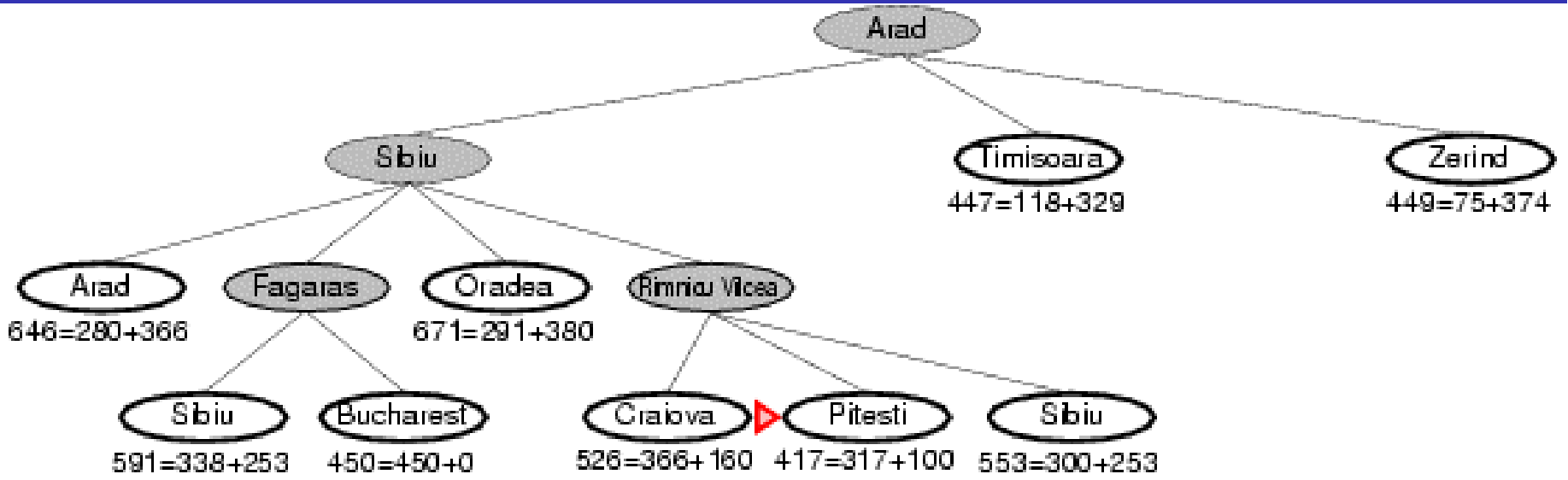
A* search example

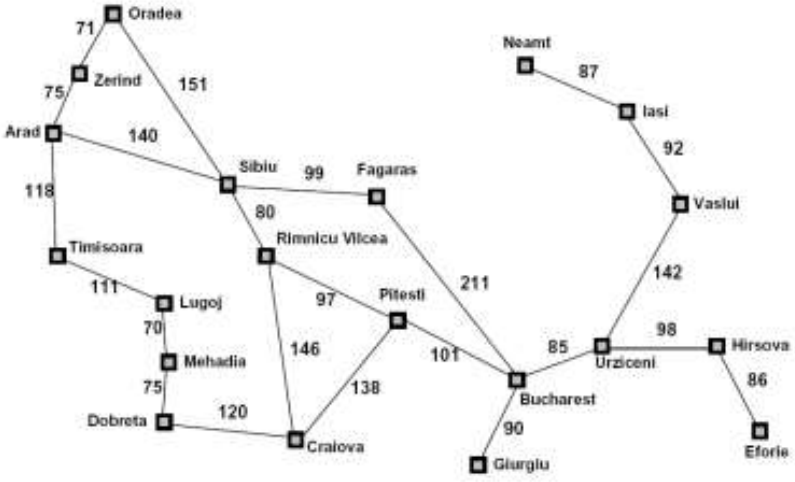




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

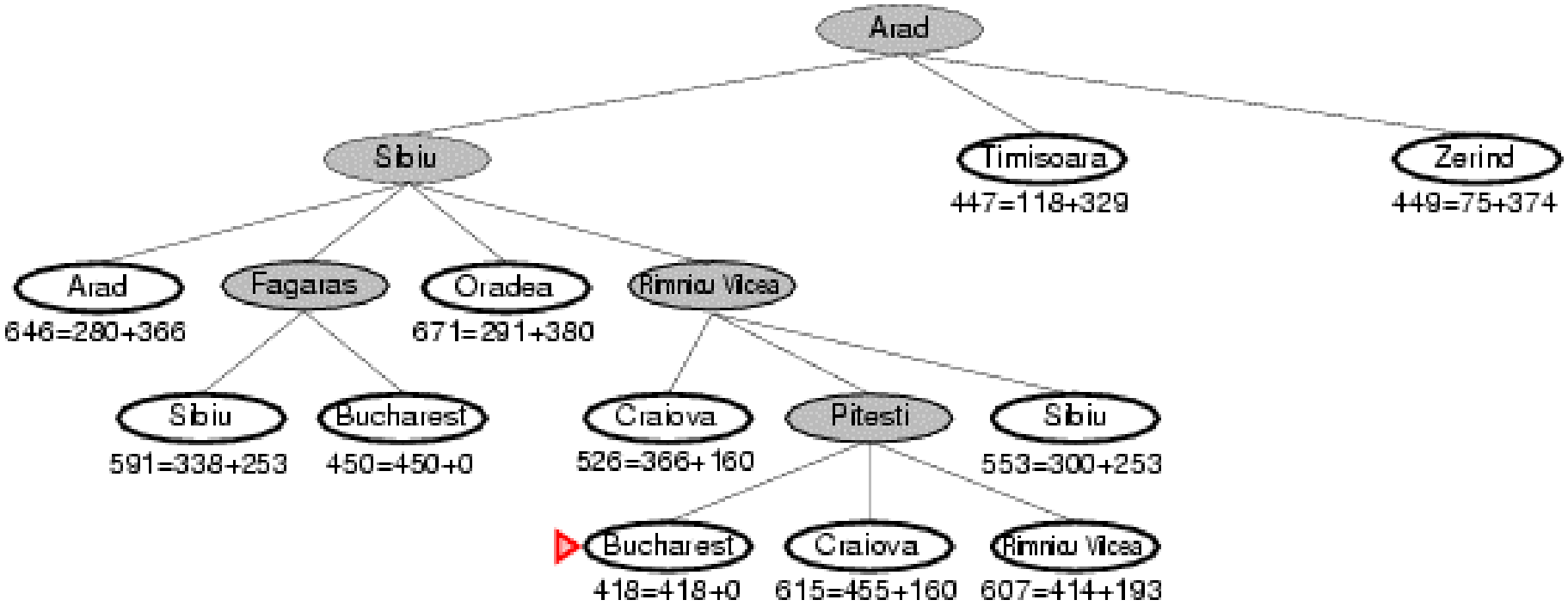
A* search example

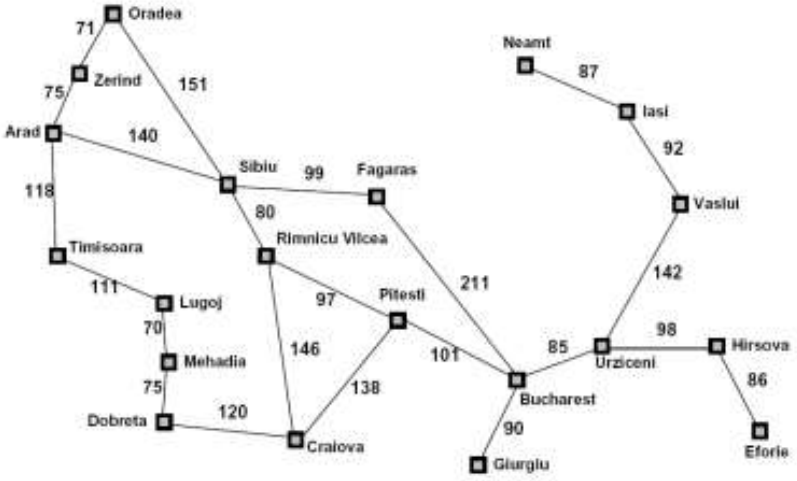




Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* search example





Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Admissible heuristics

- A heuristic *h(n)* is admissible if for every node *n*, *h(n)* <= *h*(n)*, where *h*(n)* is the true cost to reach the goal state from *n*.
- An admissible heuristic never overestimates the cost to reach the goal, i.e., it is optimistic
- Example: $h_{SLD}(n)$ (never overestimates the actual road distance)

Theorem: If *h(n)* is admissible, A* is cost-optimal.

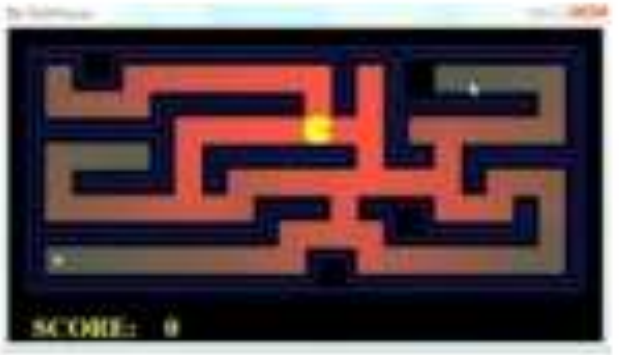
4. Properties A*

- **Complete?** Yes, as long as branching factor is always finite and every operator adds cost at least.
- **Optimal?** Yes
- **Time Complexity**: $O(b^m)$
- **Space Complexity**: $O(b^m)$

4. Comparison



Greedy



Uniform Cost



A*

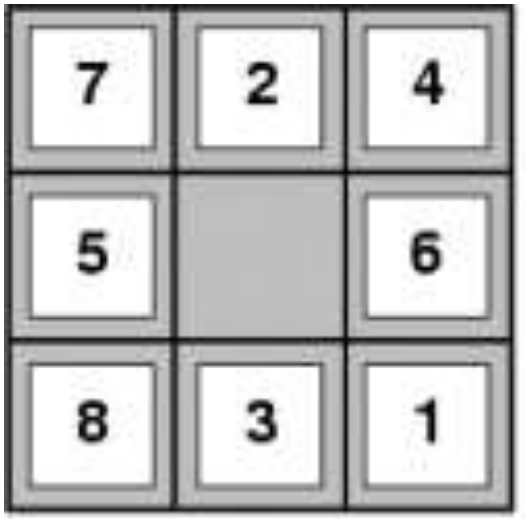
This slide from: ai.berkeley.edu

Admissible heuristics

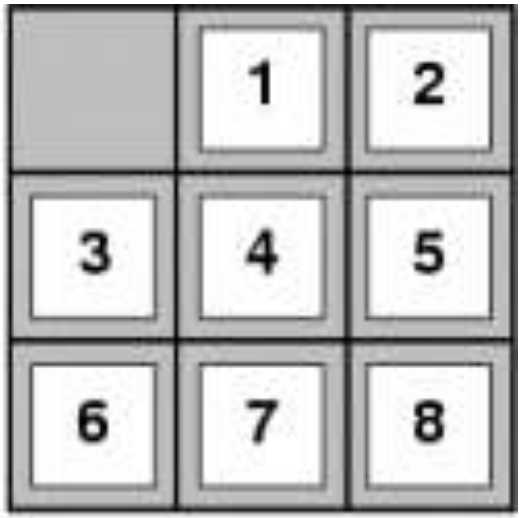
E.g., for the 8-puzzle:

- h1(n) = number of misplaced tiles
- h2(n) = total Manhattan distance (i.e., no. of squares from desired location of each tile)

- $h_1(S) = ?$
- $h_2(S) = ?$



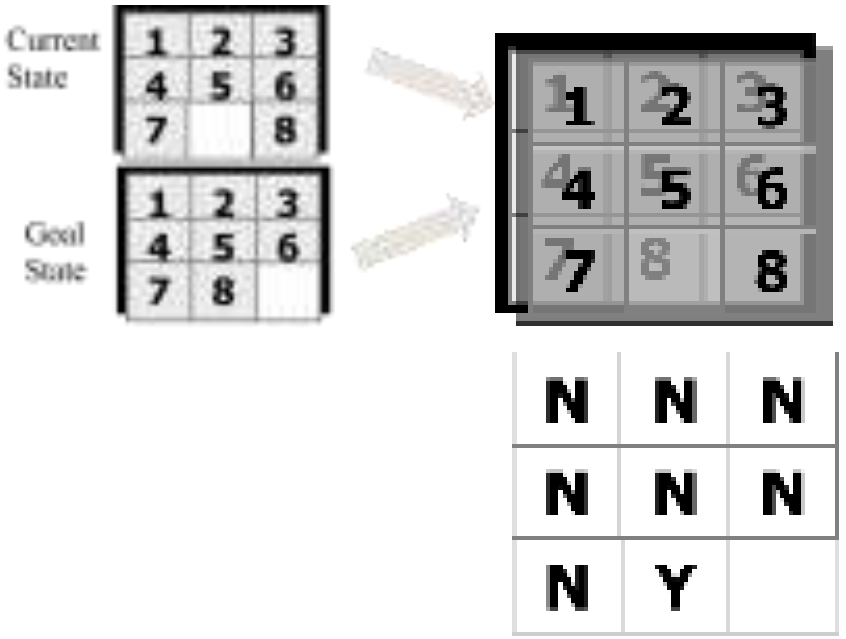
Start State



Goal State

Example: The 8-puzzle

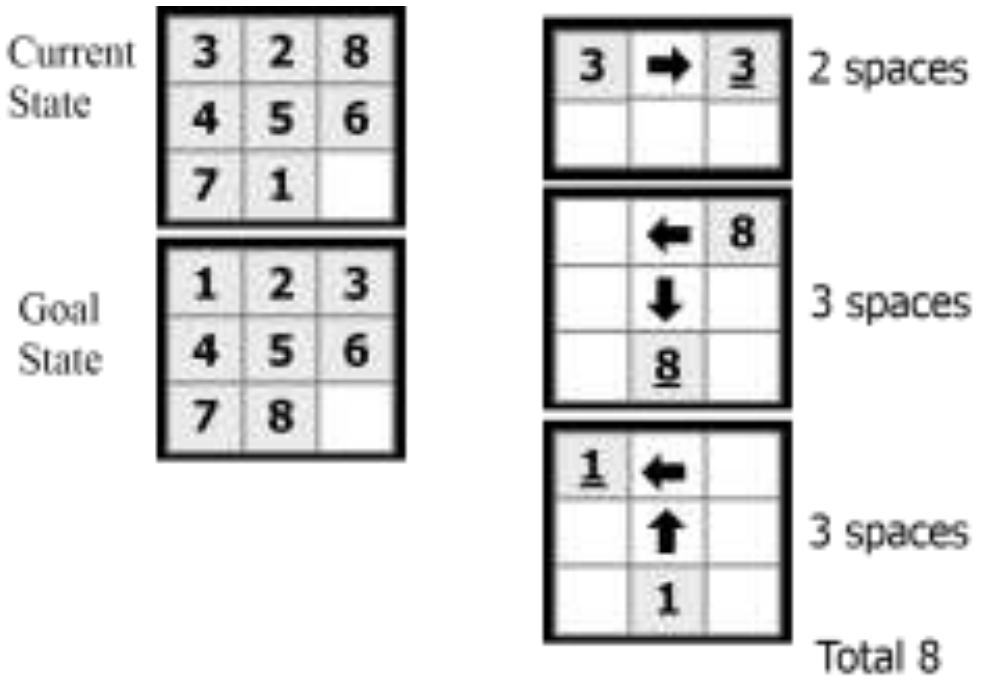
- The **number of misplaced tiles** (not including the blank)



- In this case, only 8 is misplaced, so the heuristic function evaluates to 1.
- In other words, the heuristic is telling us, that it thinks a solution might be available in just 1 more move.
- Notation: h (n) h (current state) = 1

Example: The 8-puzzle

The **Distance** (not including the blank)

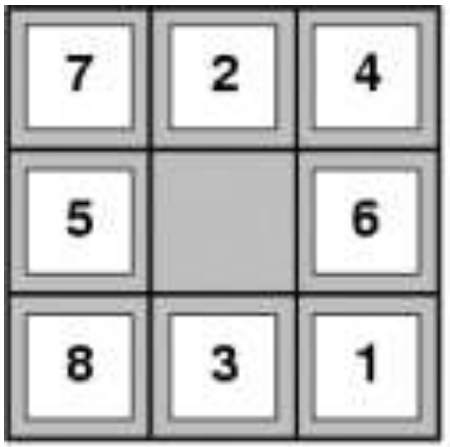


- In this case, only the "3", "8" and "1" tiles are misplaced, by 2, 3, and 3 squares respectively, so the heuristic function evaluates to 8.
- In other words, the heuristic is telling us, that it thinks a solution is available in just 8 more moves.

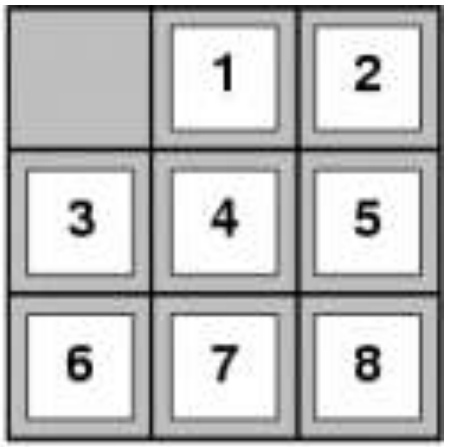
Admissible heuristics

E.g., for the 8-puzzle:

- h1(n) = number of misplaced tiles
- h2(n) = total Manhattan distance (i.e., no. of squares from desired location of each tile)



Start State



Goal State

- $h_1(S)$ = ? 8
- $h_2(S)$= ? 3+1+2+2+2+3+3+2 = 18

Admissible heuristics

- If $h_2(n) \geq h_1(n)$ for all n (both admissible) then h_2 dominates h_1. h_2 is better for search.
- Typical search costs (average number of nodes expanded):

d=12 IDS = 3,644,035 nodes
A*(h_1) = 227 nodes
A*(h_2) = 73 nodes

d=24 IDS = too many nodes
A*(h_1) = 39,135 nodes
A*(h_2) = 1,641 nodes

Admissible heuristics

- h1 and h2 are estimates of the remaining path length for the 8-puzzle, and also perfectly accurate path lengths for **simplified** versions of the puzzle.

- The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem.

- Example: In 8-puzzle, a tile can move from square X to square Y if:

  X is adjacent to Y and Y is blank,

  we can generate three relaxed problems by removing one or both of the conditions:

  (a) A tile can move from square X to square Y if X is adjacent to Y. **(h2)**

  (b) A tile can move from square X to square Y if Y is blank.

  (c) A tile can move from square X to square Y. **(h1)**

Local search algorithms

- In many optimization problems, the path to the goal is irrelevant; the goal state itself is the solution.
- State space = a set of "complete" configurations
- Example Applications:
    - Finding a valid configuration satisfying constraints, e.g., N-queens.
    - Optimization problems: finding the best state according to an objective function, e.g. job shop scheduling, portfolio management.
- In such cases, we can use local search algorithms

Local search algorithms

- LS operates by searching from a start state to neighbouring states, without keeping track of the paths, nor the set of states that have been reached.

- Hill Climbing process: the algorithm keeps track of one current state, and on each iteration moves to the neighbouring state with highest value, until reaching a "peak" where no neighbour has a higher value.

- Advantages of local search algorithms:
  - Using very little memory.
  - Finding good (sub-optimal) solutions in reasonable time for complex problems.

Hill-climbing search

Hill-climbing search algorithm is the most basic local search technique.
At each step, the current node is replaced by the best neighbor.

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
  current ← problem.INITIAL
  while true do
    neighbor ← a highest-valued successor state of current
    if VALUE(neighbor) ≤ VALUE(current) then return current
    current ← neighbor
```

VALUE(neigbour) is determined by:

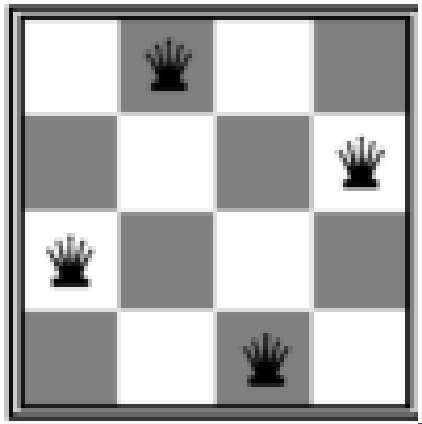
- Objective function, e.g. minimize travelling time in TSP, Or
- Heuristics to estimate the distance to goal, h(n) .

Example: n-queens

- The goal of the 8-queens problem is to place eight queens on a chessboard such that no queen attacks any other.
  - A queen attacks any piece in the same row, column or diagonal.



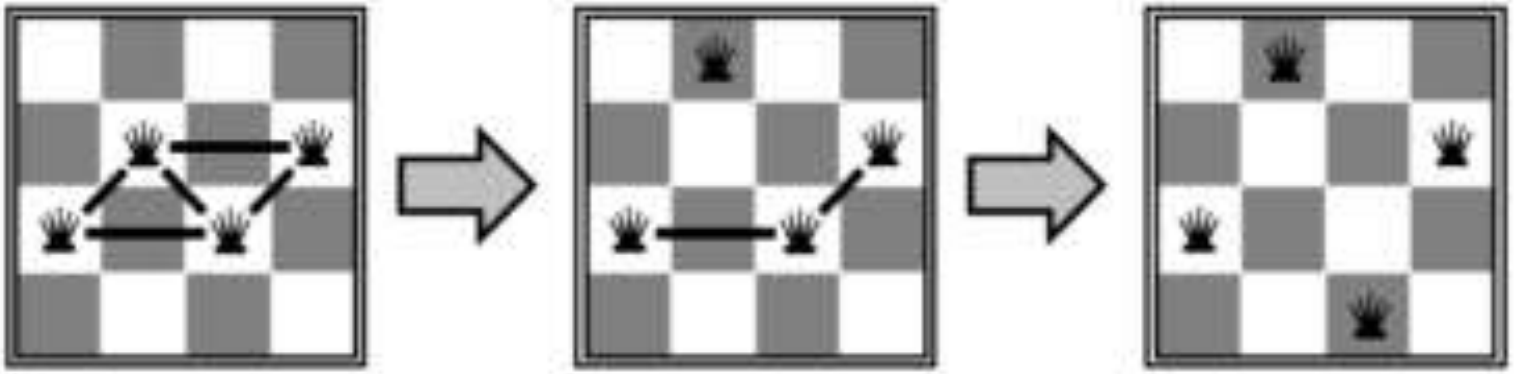
- Example of goal state:



Example: n-queens

Example: n-queens

- The **heuristic cost function *h*** is the number of pairs of queens that are attacking each other, either directly or indirectly.
- The **global minimum** of this function is *zero*, which occurs only at perfect solutions. Hill-climbing algorithms typically choose randomly among the set of best successors if there is more than one.



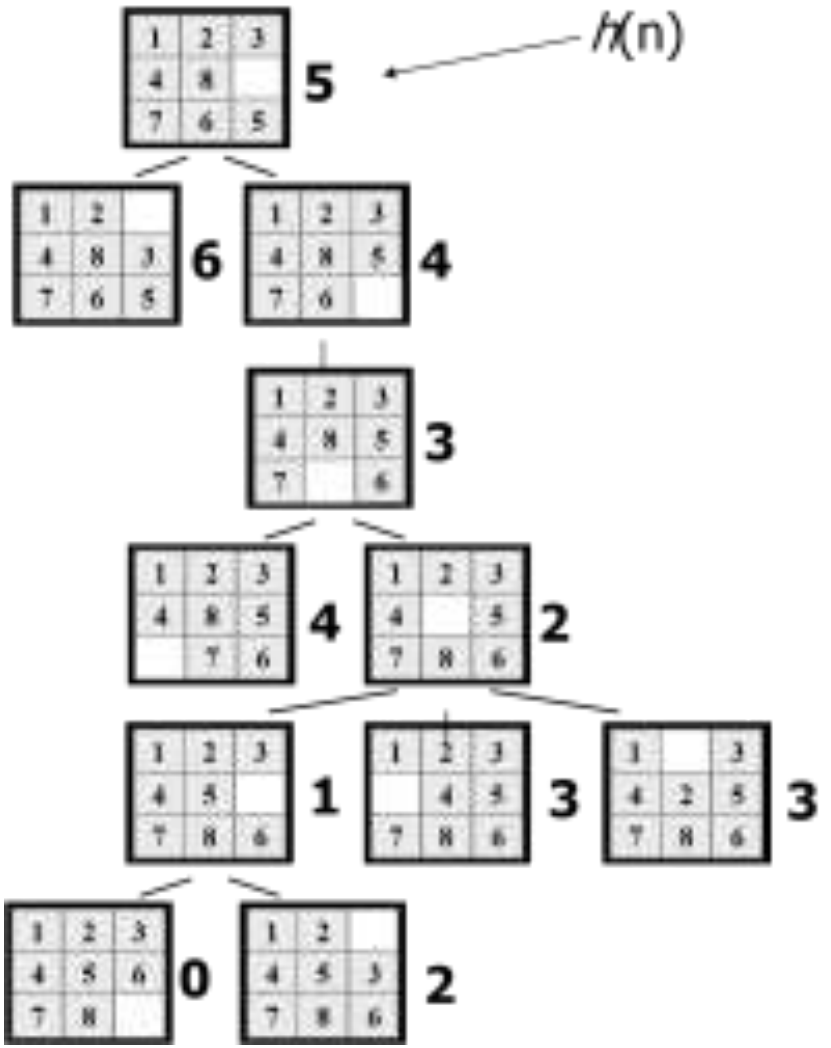
Example: n-queens

- An 8-queens state with heuristic cost estimate h = 17.
- Showing the value of h for each possible successor obtained by moving a queen within its column.
- The best moves are market.



Hill-climbing search

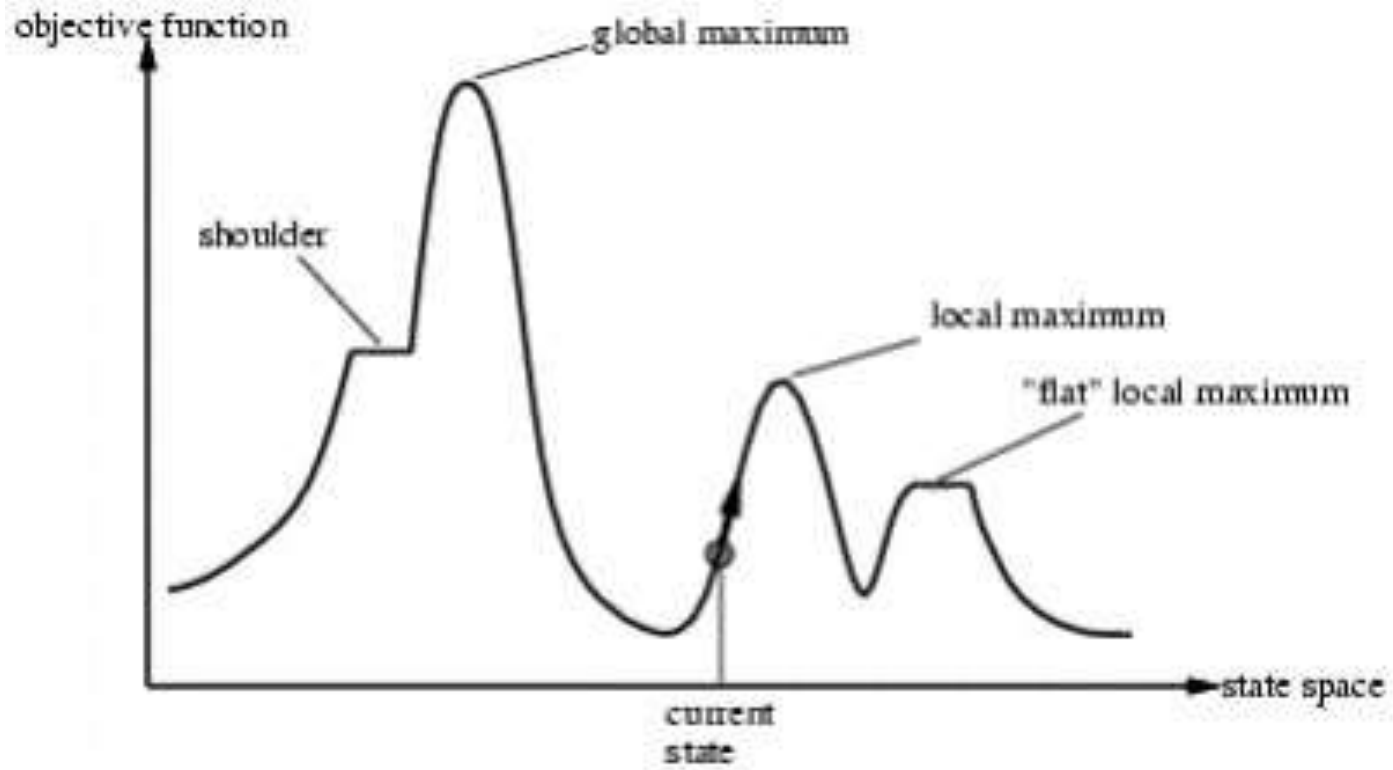
- We can use heuristics to guide hill climbing search.
- In this example, the **Manhattan Distance heuristic** helps us quickly find a solution to the 8-puzzle.



Hill-climbing search

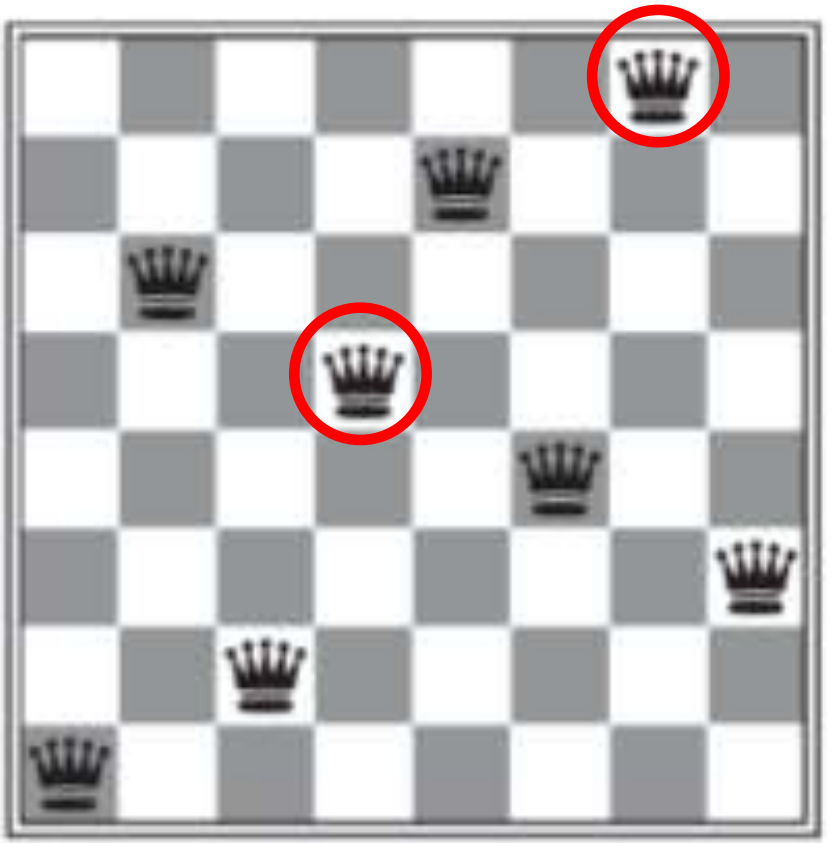
But "hill climbing has a problem..."

Problem: depending on initial state, can get stuck in local maxima.



Example: n-queens

- A local minimum in the 8-queens state space
- The state has h = 1 but every successor has a higher cost.



Hill-climbing search

Variants of hill climbing:

- First-choice hill climbing: generating successors randomly until one is generated that is better than the current state. This is a good strategy when a state has many neighbors.

- Random-restart hill climbing: conducting a series of hill-climbing searches from randomly generated initial states.

Local beam search

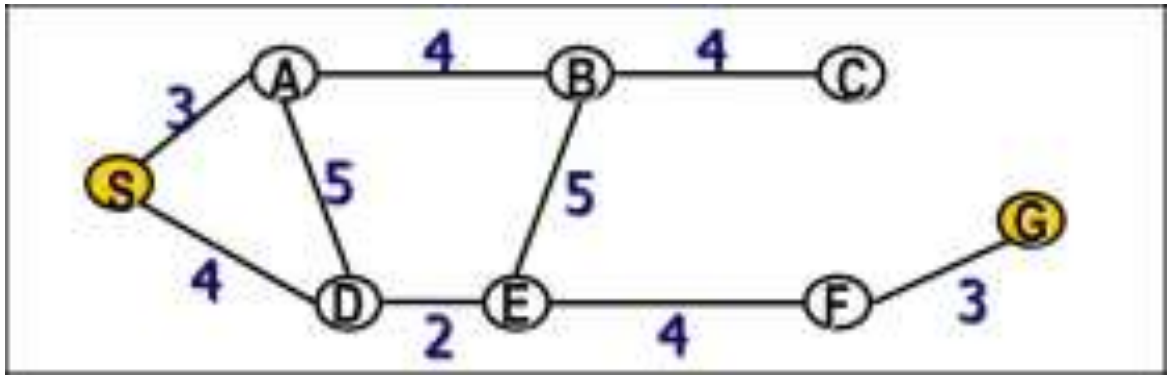
- Keep track of ***k* states** rather than just one
- Start *k* randomly generated states.
- At each iteration, all the successors of all *k* states are generated
- If any one is a goal state, stop; else select the *k* best successors from the complete list and repeat.

3. Beam Search vs breadth-first search

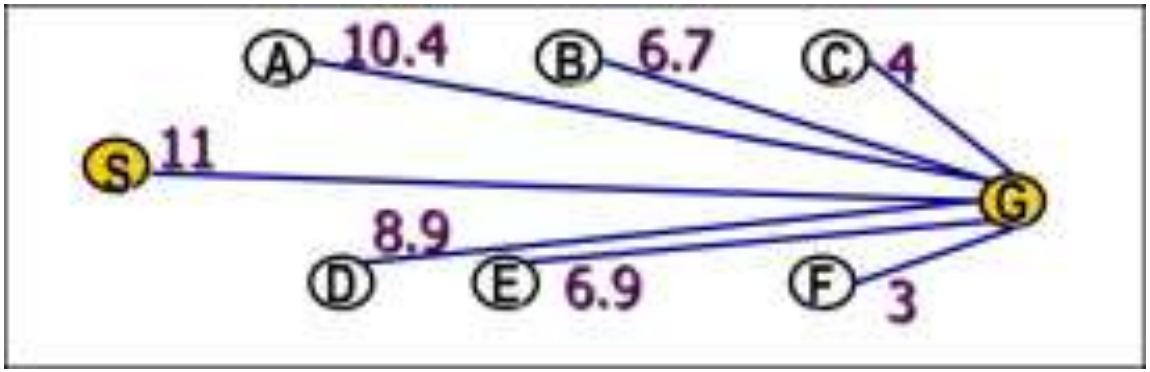
- **Narrowing** the width of the breadth-first search.
- Space and time complexity of storing and sorting the complete queue can be too inefficient in BFS.
- Beam search trims queue to the best k options (k is called the beam width) at each point.
- Focuses search more but may eliminate solutions even for finite search space graphs.

3.Beam Search (Beam)

- Let us consider the problem of finding a route on a road map:

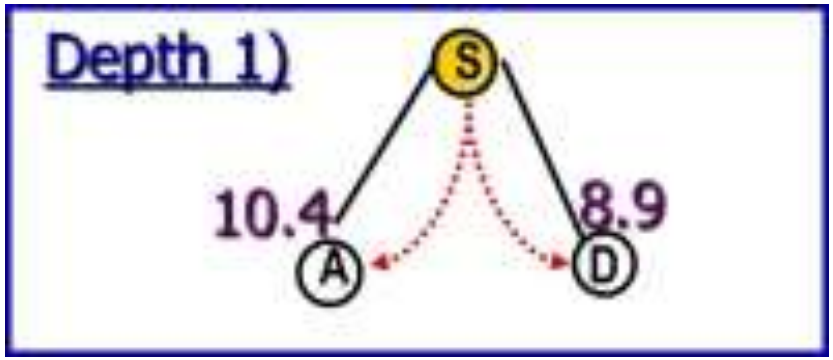


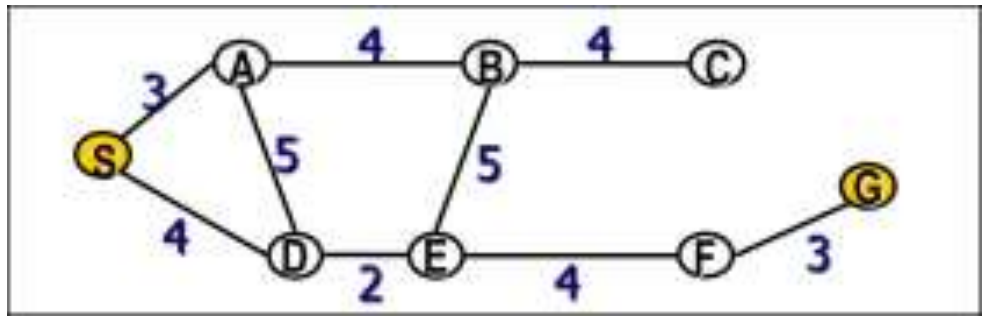
- Define f(T) = the straight-line distance from T to G

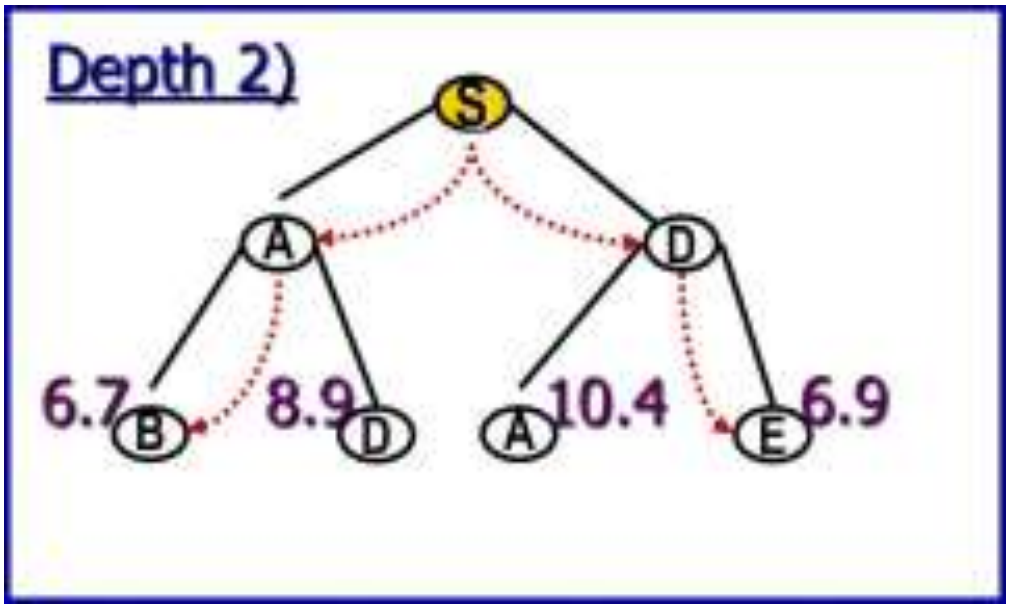


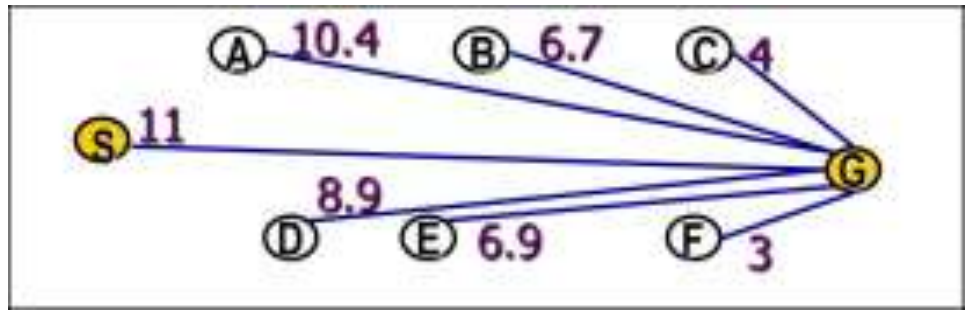
3.Beam Search (Beam)

- Assume a pre-fixed WIDTH k=2.
- Perform breadth-first, BUT: Only keep k=2 best new nodes depending on heuristic at each new level.

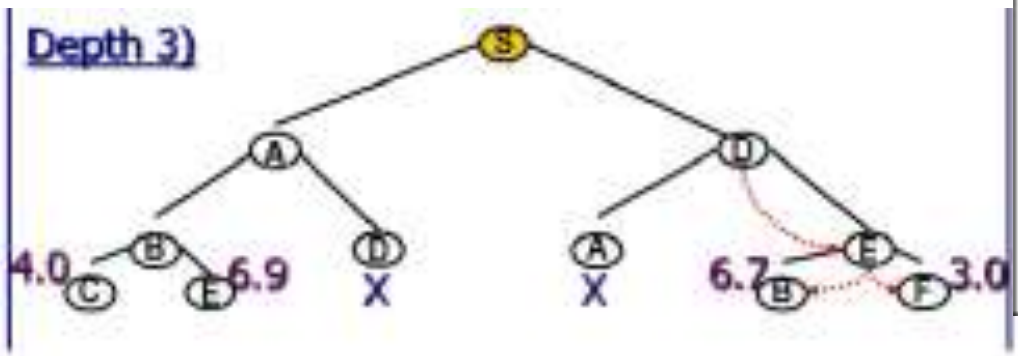


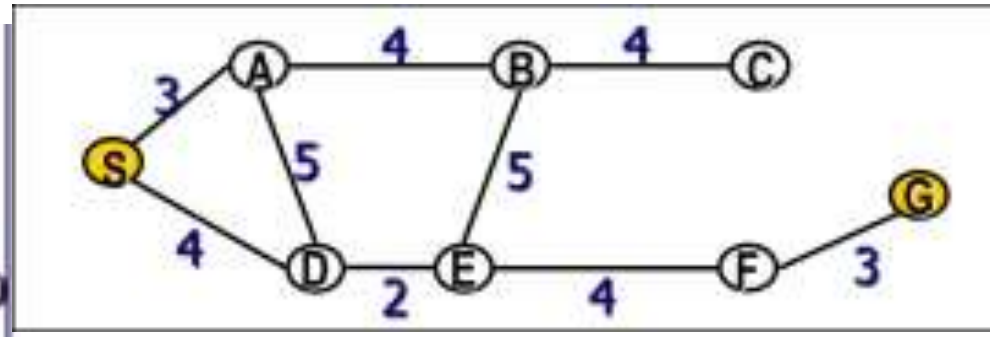


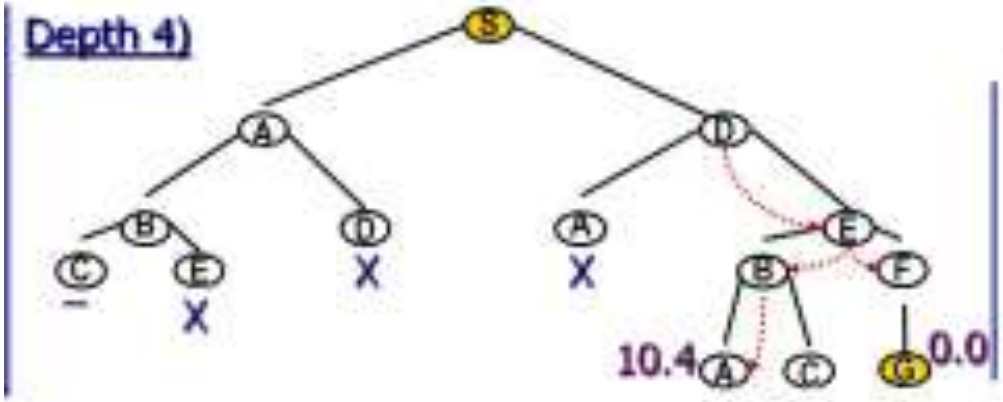


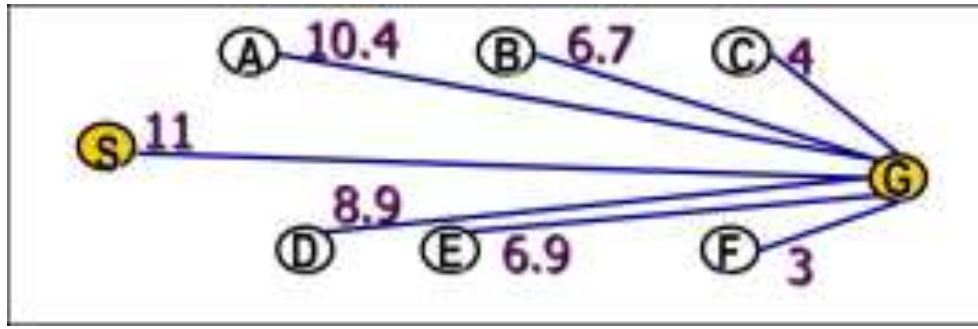


3.Beam Search (Beam)









Optimization: Ignore leaves that are not leading to goal nodes (see C).

Genetic algorithms

- A successor state is generated by combining two parent states.
- Start with *k* randomly generated states (population)
- A state is represented as a string over a finite alphabet (often a string of 0s and 1s).
- Evaluation function (fitness function in the GA terminology), fitness function should return higher values for better states.
- Produce the next generation of states by selection, crossover, and mutation

Genetic algorithms

